

Backend REST API con Node.js & Express

Sistema di Gestione Autonoleggio

Informazioni esercizio	
Livello	Intermedio
Prerequisiti	JavaScript ES6+, concetti REST, SQL (funzioni di aggregazione e stringa)
Tecnologie	Node.js 20+, Express 4, Sequelize 6, MariaDB
Endpoint richiesti	6 endpoint totali, tutti di sola lettura (GET)
Consegna	Codice sorgente, script SQL (creazione tabelle + dati), collection Postman

1. Obiettivi dell'esercizio

Lo studente dovrà progettare e implementare un backend REST di sola lettura per la gestione di un autonoleggio, usando Node.js con Express e Sequelize come ORM. A differenza degli esercizi con operazioni CRUD complete, qui non sono richieste operazioni di scrittura (POST, PUT, DELETE): l'obiettivo è concentrarsi sulla scrittura di query di lettura evolute, che sfruttano funzioni SQL di concatenazione, conversione e aggregazione richiamate direttamente nella SELECT tramite `sequelize.fn` e `sequelize.literal`.

Al termine dell'esercizio lo studente sarà in grado di:

- Definire modelli Sequelize con una relazione multi-a-molti tramite tabella associativa con attributi propri
- Utilizzare `sequelize.fn` e `sequelize.literal` per richiamare funzioni SQL native (CONCAT, DATE_FORMAT, TIMESTAMPDIFF, DATEDIFF, FORMAT, ROUND)
- Restituire campi calcolati e formattati nelle risposte JSON, senza modificarli lato JavaScript
- Scrivere query di aggregazione con COUNT, SUM, AVG e GROUP BY
- Effettuare ricerche testuali case-insensitive con l'operatore LIKE
- Produrre script SQL per la creazione delle tabelle e il popolamento dei dati di test

2. Struttura del database

Il database MariaDB deve contenere due tabelle principali collegate da una relazione multi-a-molti, gestita tramite una tabella associativa che, oltre alle due chiavi esterne, contiene anche attributi propri (le date del noleggio e la valutazione del cliente).

2.1 Tabella principale — Cliente

Rappresenta un cliente registrato presso l'autonoleggio.

Campo	Tipo	Vincoli	Note
id	INTEGER	PK, AUTOINCREMENT	Chiave primaria autoincrementale

Campo	Tipo	Vincoli	Note
nome	STRING	NOT NULL	Nome del cliente
cognome	STRING	NOT NULL	Cognome del cliente
email	STRING	UNIQUE, NOT NULL	Indirizzo email del cliente
telefono	STRING	opzionale	Numero di telefono
dataNascita	DATEONLY	NOT NULL	Data di nascita, usata per calcolare l'età
numeroPatente	STRING	opzionale	Numero della patente di guida
dataIscrizione	DATE	default NOW	Data e ora di registrazione al servizio

2.2 Tabella secondaria — Auto (relazione Multi-a-Molti con Cliente)

Rappresenta un'automobile della flotta. Un cliente può aver noleggiato molte auto nel tempo; una stessa auto può essere stata noleggiata da molti clienti.

In Sequelize: `Cliente.belongsToMany(Auto, { through: 'Noleggio' })` e `Auto.belongsToMany(Cliente, { through: 'Noleggio' })`.

Campo	Tipo	Vincoli	Note
id	INTEGER	PK, AUTOINCREMENT	Chiave primaria autoincrementale
targa	STRING	UNIQUE, NOT NULL	Targa del veicolo, es. "AA123BB"
marca	STRING	NOT NULL	Es. "Fiat", "Volkswagen", "Audi"
modello	STRING	NOT NULL	Es. "Panda", "Golf", "A3"
prezzoGiornaliero	FLOAT	NOT NULL	Costo di noleggio al giorno in euro
annoImmatricolazione	INTEGER	NOT NULL	Anno di immatricolazione
cilindrata	INTEGER	NOT NULL	Cilindrata del motore in cc

2.3 Tabella associativa — Noleggio

A differenza di una semplice tabella pivot, questa tabella associativa contiene anche dati propri del noleggio, e non solo le due chiavi esterne.

Campo	Tipo	Vincoli	Note
ClienteId	INTEGER	PK composta, FK → Cliente	Chiave esterna verso Cliente
AutoId	INTEGER	PK composta, FK → Auto	Chiave esterna verso Auto
dataRitiro	DATEONLY	NOT NULL	Data di ritiro del veicolo
dataRiconsegna	DATEONLY	NOT NULL	Data di riconsegna del veicolo
valutazione	INTEGER	opzionale, 1-5	Valutazione del servizio da parte del cliente

Nota sul diagramma ER

Prima di scrivere codice, disegnare il diagramma ER delle due tabelle principali e della tabella associativa Noleggio, evidenziando che quest'ultima ha una chiave primaria composta (ClienteId + AutoId) e tre colonne aggiuntive che non sono semplici chiavi esterne.

3. Tecnologie e struttura del progetto

3.1 Stack tecnologico richiesto

- Node.js 20 LTS o superiore
- Express 4 — framework web per la creazione di REST API
- Sequelize 6 — ORM per Node.js compatibile con MariaDB, MySQL, PostgreSQL
- mariadb — driver nativo Node.js per MariaDB (alternativa: mysql2)
- dotenv — gestione delle variabili di ambiente per le credenziali del database

3.2 Struttura consigliata del progetto

```

autonoleggio/
├── app.js                # entry point Express
├── config/
│   └── database.js      # connessione Sequelize + MariaDB
├── models/
│   ├── index.js        # carica modelli e associazioni
│   ├── Cliente.js      # modello Sequelize Cliente
│   ├── Auto.js         # modello Sequelize Auto
│   └── Noleggio.js     # modello Sequelize della pivot
├── routes/
│   ├── clienti.js      # router endpoint clienti
│   ├── auto.js         # router endpoint auto
│   └── statistiche.js  # router endpoint statistiche
├── sql/
│   ├── create_tables.sql # script di creazione tabelle
│   └── seed_data.sql    # script di popolamento dati
├── postman/
│   └── autonoleggio.json # collection Postman
├── .env                # credenziali DB (non committare!)
├── package.json
└── README.md

```

4. Elenco degli endpoint richiesti

Il progetto deve esporre esattamente 6 endpoint, tutti di tipo GET. Ogni endpoint deve restituire codice HTTP 200 in caso di successo, 404 se la risorsa richiesta non esiste, e rispondere sempre con JSON. Ogni endpoint richiede l'utilizzo di almeno una funzione SQL di concatenazione, conversione o aggregazione all'interno della query.

#	Metodo	URL	Descrizione	Tag
1	GET	/api/clienti	Lista clienti con nome completo concatenato	Cliente
2	GET	/api/clienti/:id	Dettaglio cliente con età calcolata e auto noleggiate	Cliente
3	GET	/api/clienti/ricerca	Ricerca clienti per nome o cognome (?q=...)	Cliente
4	GET	/api/auto	Lista auto con modello e prezzo formattati	Auto
5	GET	/api/auto/:id	Dettaglio auto con numero noleggi e valutazione media	Auto
6	GET	/api/statistiche/auto-piu-noleggiate	Classifica auto per noleggi e incasso	Statistiche

5. Specifiche dettagliate degli endpoint

Endpoint 1 — GET /api/clienti

Deve restituire l'elenco dei clienti includendo un campo calcolato nomeCompleto ottenuto concatenando nome e cognome lato database, non lato JavaScript. Usare `sequelize.fn` e `sequelize.col`:

```
const clienti = await Cliente.findAll({
  attributes: [
    'id', 'email', 'telefono',
    [sequelize.fn('CONCAT', sequelize.col('nome'), ' ',
      sequelize.col('cognome')), 'nomeCompleto'],
    [sequelize.fn('DATE_FORMAT', sequelize.col('dataIscrizione'),
      '%d/%m/%Y'), 'dataIscrizioneFormattata']
  ]
});
```

Endpoint 2 — GET /api/clienti/:id

Deve restituire il dettaglio del cliente con un campo calcolato eta, ottenuto dalla differenza in anni tra la data odierna e dataNascita tramite la funzione SQL TIMESTAMPDIFF, e l'elenco delle auto noleggiate (tramite include su Auto, includendo anche gli attributi della tabella Noleggio: dataRitiro, dataRiconsegna e valutazione).

```
attributes: {
  include: [
    [sequelize.fn('TIMESTAMPDIFF', sequelize.literal('YEAR'),
      sequelize.col('Cliente.dataNascita'),
      sequelize.fn('NOW')), 'eta']
  ]
}
```

Se il cliente non esiste, restituire HTTP 404 con il messaggio: "Cliente non trovato."

Endpoint 3 — GET /api/clienti/ricerca?q=...

Deve effettuare una ricerca case-insensitive su nome e cognome usando l'operatore Op.like, restituendo comunque il campo nomeCompleto concatenato come nell'endpoint 1. Esempio: /api/clienti/ricerca?q=ross deve trovare sia "Rossi" che "Rossetti". Se il parametro q non viene fornito, restituire HTTP 400 con il messaggio: "Parametro di ricerca 'q' obbligatorio."

Attenzione all'ordine delle route!

In Express, /api/clienti/ricerca deve essere definita PRIMA di /api/clienti/:id nel file delle route, altrimenti Express interpreterà 'ricerca' come un valore dinamico di :id.

Endpoint 4 — GET /api/auto

Deve restituire l'elenco delle auto con due campi calcolati: modelloCompleto (concatenazione di marca e modello) e prezzoFormattato (prezzo con due decimali concatenato al simbolo dell'euro tramite CONCAT e FORMAT).

```
[sequelize.fn('CONCAT', sequelize.col('marca'), ' ',
  sequelize.col('modello')), 'modelloCompleto'],
[sequelize.literal("CONCAT(FORMAT(prezzoGiornaliero, 2), ' €')"),
  'prezzoFormattato']
```

Endpoint 5 — GET /api/auto/:id

Deve restituire il dettaglio dell'auto con il numero totale di noleggi ricevuti (COUNT) e la valutazione media arrotondata a un decimale (AVG + ROUND), oltre all'elenco dei clienti che l'hanno noleggiata. Se l'auto non esiste, restituire HTTP 404 con il messaggio: "Auto non trovata."

Endpoint 6 — GET /api/statistiche/auto-piu-noleggiate

Deve restituire una classifica delle auto ordinata per numero di noleggi decrescente, con i campi: `modelloCompleto`, `numeroNoleggi` (COUNT), `incassoTotale` (SUM dei giorni di noleggio moltiplicati per il prezzo giornaliero, usando `DATEDIFF`) e `valutazioneMedia` (AVG arrotondata). Richiede una query con `GROUP BY` sull'auto e `ORDER BY` sul conteggio.

```
[sequelize.literal(
  'SUM(DATEDIFF(dataRiconsegna, dataRitiro) * prezzoGiornaliero)'
), 'incassoTotale']
```

6. Definizione dei modelli e delle associazioni

6.1 Associazioni con attributi sulla tabella pivot

Poiché la tabella `Noleggio` contiene colonne proprie, è necessario definirla come modello Sequelize a sé stante e usarla come `through` esplicito, anziché lasciare che Sequelize la generi automaticamente:

```
Cliente.belongsToMany(Auto, { through: Noleggio });
Auto.belongsToMany(Cliente, { through: Noleggio });
```

In questo modo, includendo `Auto` in una query su `Cliente`, sarà possibile accedere anche ai campi `dataRitiro`, `dataRiconsegna` e `valutazione` tramite l'oggetto `Noleggio` annidato in ogni risultato.

6.2 Include con attributi della tabella pivot

```
const cliente = await Cliente.findByPk(id, {
  include: [{
    model: Auto,
    through: {
      attributes: ['dataRitiro', 'dataRiconsegna', 'valutazione']
    }
  }]
});
```

7. Script SQL richiesti

Nella cartella `sql/` devono essere presenti due file SQL eseguibili direttamente su MariaDB.

7.1 create_tables.sql

Deve contenere le istruzioni `CREATE TABLE` per le tre tabelle (`Cliente`, `Auto`, `Noleggio`), con i vincoli di chiave primaria (composta per `Noleggio`), chiave esterna e `NOT NULL`. Usare `IF NOT EXISTS` per rendere lo script rieseguibile. Le tabelle `Cliente` e `Auto` vanno create prima di `Noleggio`, che le riferenzia con le chiavi esterne.

7.2 seed_data.sql

Deve contenere le istruzioni `INSERT INTO` per popolare il database con dati di test realistici. Sono richiesti almeno:

- 10 clienti con date di nascita ed email diverse
- 8 auto con marche, prezzi e cilindrata variati
- almeno 20 noleggi distribuiti tra clienti e auto, con date e valutazioni diverse, in modo che le query di aggregazione dell'endpoint 6 restituiscano risultati significativi (alcune auto molto noleggiate, altre poco)

8. Requisiti di consegna

Il materiale da consegnare è composto da tre elementi obbligatori:

- 1 Codice sorgente Node.js — Cartella del progetto completa e funzionante, con file `package.json` contenente tutte le dipendenze e uno script "start" per avviare il server. Il codice deve essere privo di errori e rispettare la struttura delle cartelle indicata al punto 3.2.
- 2 Script SQL — I due file `create_tables.sql` e `seed_data.sql` nella cartella `sql/`. Gli script devono essere eseguibili su un server MariaDB pulito senza alcuna modifica manuale.
- 3 Collection Postman — File `.json` importabile in Postman con tutti e 6 gli endpoint configurati, organizzati in cartelle (Clienti, Auto, Statistiche) che rispecchino i tag degli endpoint. Includere, dove utile, esempi di query string (es. `?q=ross`).

Avvertenza

L'uso di soluzioni generate automaticamente da strumenti di intelligenza artificiale senza comprensione del codice prodotto è considerato disonestà accademica. Il docente si riserva di richiedere la spiegazione di qualsiasi parte del materiale consegnato.